

20 Production Business-Logic Patterns

System Design / Lead Engineer Interview Notes

Core idea: Production systems are not just CRUD. Strong system design separates business state, side effects, reliability, consistency, and communication.

1. State Change $\neq$ Notification

A user reads a message: **MessageRead** $\rightarrow$ **update read_at** $\rightarrow$ **don't notify sender**. A new message may notify, but a read event normally does not. The key idea is that not every state transition creates a user-facing notification.

2. Delete $\neq$ Permanently Delete

A Delete action can mean **soft_deleted = true** rather than physical removal. This supports recovery, auditability, and retention requirements.

3. Cancel $\neq$ Delete

An order that is cancelled should usually remain in the database. Payment, refund, customer, timestamps, and audit history may still depend on the order record.

4. Unfollow $\neq$ Remove History

Removing a social relationship does not necessarily erase previous comments, likes, messages, or other historical data. Relationship state and historical data are separate concerns.

5. Payment Success $\neq$ Order Completion

Payment success can be one step in a larger workflow: **Payment SUCCESS** $\rightarrow$ **order processing** $\rightarrow$ **inventory confirmation** $\rightarrow$ **order CONFIRMED**. Payment alone may not complete the business transaction.

6. API Success $\neq$ Business Success

HTTP 200 OK only describes the API request/response outcome. The business operation may still be pending, for example: **200 OK + status: PENDING**.

7. Request Accepted $\neq$ Request Completed

Heavy work can return **202 Accepted**, then continue through a queue and worker: **API** $\rightarrow$ **Queue** $\rightarrow$ **Worker** $\rightarrow$ **Result**. Avoid blocking an HTTP request unnecessarily.

8. Retry $\neq$ Duplicate Operation

A network timeout can cause a client to retry the same operation. Use an **Idempotency-Key** when the business operation must not execute twice, especially for payments and bookings.

9. Timeout ≠ Failure

If a payment request times out, the client may not know whether the payment actually succeeded. Treat timeout as an uncertain state and use status lookup, webhook, or reconciliation mechanisms.

10. Cache Miss ≠ Database Failure

A Redis cache miss is normal: **Redis MISS** → **DB** → **optionally populate cache**. Redis being unavailable is a different failure scenario and may require fallback behavior.

11. Logout ≠ Automatic JWT Invalidation

A stateless JWT can remain technically valid until expiry unless the system uses short-lived access tokens, refresh-token revocation, a denylist, sessions, or another invalidation strategy.

12. User Deleted ≠ Immediately Delete Everything

Deleting an account does not necessarily mean immediately deleting every related record. Systems may need PII anonymization, retention rules, financial records, or audit history.

13. Inventory Reservation ≠ Inventory Deduction

During checkout, stock may first be reserved, then finalized after payment/order confirmation. Reservation helps prevent multiple users from buying the same limited inventory.

14. Notification ≠ Source of Truth

A notification saying 'Payment successful' should not itself be treated as proof that payment succeeded. The payment system/database is the source of truth; notifications are a communication mechanism.

15. Email Sent ≠ Email Delivered

Email lifecycle states can differ: **QUEUED** → **SENT/ACCEPTED** → **DELIVERED**, with possible **BOUNCED/FAILED** states. Sending to a provider is not the same as delivery to the recipient.

16. Database Transaction ≠ Distributed Transaction

A transaction inside one database does not automatically cover multiple services. For distributed workflows, patterns such as **Outbox**, **Saga**, **retries**, **compensation**, and **idempotency** may be needed.

17. Online ≠ Currently Using the App

An 'online' indicator can be based on a recent heartbeat. A timestamp such as **last_seen_at** is often more informative than a simple boolean because presence expires over time.

18. Duplicate Request ≠ Duplicate Business Operation

A user may perform one action while the network produces multiple HTTP requests. Backend design should distinguish an HTTP request from the underlying business operation, often using idempotency keys.

19. Event Exists ≠ Event Must Trigger Side Effects

An internal event such as **ConnectionRemoved** can update state and audit history without sending a notification. Events are facts; side effects are business rules applied to those facts.

20. Eventually Consistent $\neq$ Incorrect

After a write, a search index, cache, or replica may briefly lag behind the primary database. During that window, different components can show different states without the system necessarily being broken.

The Mental Model

When designing any production feature, ask these 8 questions before choosing technologies:

1. What is the business state?
2. What changes that state?
3. What is the source of truth?
4. Does this action need a notification?
5. Can the request be retried?
6. Can the operation happen twice?
7. What happens if a downstream service fails?
8. Does this need to happen immediately or asynchronously?

The Second Skill

Don't think only: "I know Node.js/React."

Think: "I can design and reason about production systems."

That second skill means you can reason about business rules, failure modes, retries, idempotency, asynchronous processing, source of truth, consistency, notifications, and distributed-system trade-offs—not just write the API or UI.

Interview Shortcut

For almost any system-design problem, mentally walk through: **State** → **Source of Truth** → **Event** → **Side Effect** → **Failure** → **Retry** → **Idempotency** → **Async/Sync** → **Consistency**.